

**LAPORAN PRAKTIKUM  
PEMROGRAMAN MOBILE  
MODUL 1**



**ANDROID BASICS WITH COMPOSE**

**Oleh:**

**Nazla Salsabila**

**NIM. 2410817320001**

**PROGRAM STUDI TEKNOLOGI INFORMASI  
FAKULTAS TEKNIK  
UNIVERSITAS LAMBUNG MANGKURAT  
MARET 2026**

**LEMBAR PENGESAHAN**  
**LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE**  
**MODUL 1**

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basics with Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nazla Salsabila  
NIM : 2410817320001

Menyetujui,  
Asisten Praktikum

Mengetahui,  
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan  
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,  
M.Kom.  
NIP. 199710312025061009

## DAFTAR ISI

|                         |    |
|-------------------------|----|
| LEMBAR PENGESAHAN ..... | 2  |
| DAFTAR ISI.....         | 3  |
| DAFTAR GAMBAR.....      | 4  |
| DAFTAR TABEL.....       | 5  |
| SOAL 1 .....            | 6  |
| A. Source Code .....    | 8  |
| B. Output Program.....  | 14 |
| C. Pembahasan.....      | 17 |
| SOAL 2 .....            | 20 |
| A. Pembahasan.....      | 20 |
| SOAL 3 .....            | 21 |
| A. Pembahasan.....      | 21 |
| SOAL 4 .....            | 22 |
| A. Pembahasan.....      | 22 |
| SOAL 5 .....            | 23 |
| A. Pembahasan.....      | 23 |
| DAFTAR PUSTAKA .....    | 24 |
| TAUTAN GIT .....        | 25 |

## DAFTAR GAMBAR

|                                                        |    |
|--------------------------------------------------------|----|
| Gambar 1. Tampilan Awal Aplikasi .....                 | 6  |
| Gambar 2. Tampilan Dadu Setelah Di-Roll.....           | 7  |
| Gambar 3. Tampilan Roll Dadu Double .....              | 7  |
| Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose..... | 14 |
| Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose..... | 14 |
| Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose..... | 15 |
| Gambar 7. Screenshot Hasil Jawaban Soal 1 XML .....    | 15 |
| Gambar 8. Screenshot Hasil Jawaban Soal 1 XML .....    | 16 |
| Gambar 9. Screenshot Hasil Jawaban Soal 1 XML .....    | 16 |

## DAFTAR TABEL

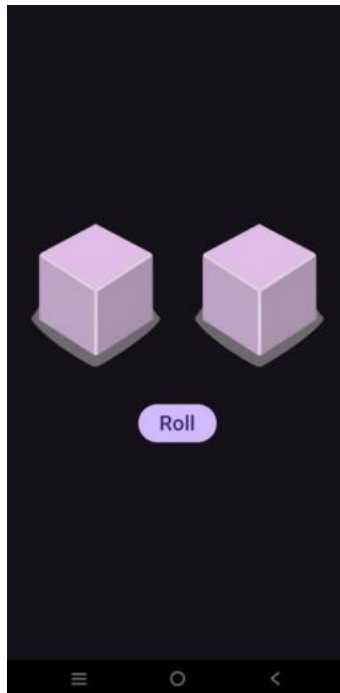
|                                                           |    |
|-----------------------------------------------------------|----|
| Tabel 1. Source Code MainActivity.kt XML Soal 1 .....     | 8  |
| Tabel 2. Source Code MainActivity.kt Compose Soal 1 ..... | 9  |
| Tabel 3. Source Code Activity_Main.XML Soal 1 .....       | 12 |

## SOAL 1

### Soal Praktikum:

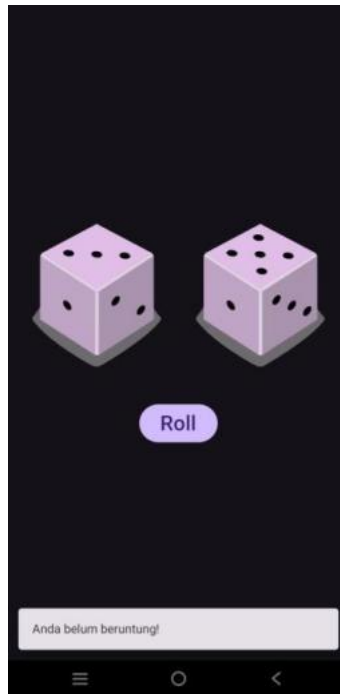
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



*Gambar 1. Tampilan Awal Aplikasi*

2. Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti yang dapat dilihat pada Gambar 2.



*Gambar 2. Tampilan Dadu Setelah Di-Roll*

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti yang dapat dilihat pada Gambar 3.



*Gambar 3. Tampilan Roll Dadu Double*

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut:  
[https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd\\_9SgFh8kw8X9ySm/view?usp=sharing](https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing)

#### A. Source Code

##### MainActivity.kt XML

*Tabel 1. Source Code MainActivity.kt XML Soal 1*

|    |                                                      |
|----|------------------------------------------------------|
| 1  | package com.nazlasalsabila.dicerollxml               |
| 2  |                                                      |
| 3  | import android.os.Bundle                             |
| 4  | import android.widget.*                              |
| 5  | import androidx.appcompat.app.AppCompatActivity      |
| 6  |                                                      |
| 7  | class MainActivity : AppCompatActivity() {           |
| 8  |                                                      |
| 9  | private lateinit var dice1: ImageView                |
| 10 | private lateinit var dice2: ImageView                |
| 11 | private lateinit var btnRoll: Button                 |
| 12 | private lateinit var tvResult: TextView              |
| 13 |                                                      |
| 14 | override fun onCreate(savedInstanceState: Bundle?) { |
| 15 | super.onCreate(savedInstanceState)                   |
| 16 | setContentView(R.layout.activity_main)               |
| 17 |                                                      |
| 18 | dice1 = findViewById(R.id.dice1)                     |
| 19 | dice2 = findViewById(R.id.dice2)                     |
| 20 | btnRoll = findViewById(R.id.btnRoll)                 |
| 21 | tvResult = findViewById(R.id.tvResult)               |
| 22 |                                                      |
| 23 | btnRoll.setOnClickListener {                         |

|    |                                                   |
|----|---------------------------------------------------|
| 24 | rollDice()                                        |
| 25 | }                                                 |
| 26 | }                                                 |
| 27 |                                                   |
| 28 | private fun rollDice() {                          |
| 29 | val result1 = (1..6).random()                     |
| 30 | val result2 = (1..6).random()                     |
| 31 |                                                   |
| 32 | updateDiceImage(dice1, result1)                   |
| 33 | updateDiceImage(dice2, result2)                   |
| 34 |                                                   |
| 35 | tvResult.text = if (result1 == result2) {         |
| 36 | "Selamat, anda dapat dadu double!"                |
| 37 | } else {                                          |
| 38 | "Anda belum beruntung!"                           |
| 39 | }                                                 |
| 40 | }                                                 |
| 41 |                                                   |
| 42 | private fun updateDiceImage(imageView: ImageView, |
| 43 | value: Int) {                                     |
| 44 | val imageRes = when (value) {                     |
| 45 | 1 -> R.drawable.dice_1                            |
| 46 | 2 -> R.drawable.dice_2                            |
| 47 | 3 -> R.drawable.dice_3                            |
| 48 | 4 -> R.drawable.dice_4                            |
| 49 | 5 -> R.drawable.dice_5                            |
| 50 | else -> R.drawable.dice_6                         |
| 51 | }                                                 |
| 52 | imageView.setImageResource(imageRes)              |
| 53 | }                                                 |

## MainActivity.kt Compose

*Tabel 2. Source Code MainActivity.kt Compose Soal 1*

|   |                                               |
|---|-----------------------------------------------|
| 1 | package com.nazlasalsabila.dicerollxml        |
| 2 |                                               |
| 3 | import android.os.Bundle                      |
| 4 | import androidx.activity.ComponentActivity    |
| 5 | import androidx.activity.compose.setContent   |
| 6 | import androidx.compose.foundation.Image      |
| 7 | import androidx.compose.foundation.background |
| 8 | import androidx.compose.foundation.layout.*   |

```

9  import androidx.compose.material3.*
10 import androidx.compose.runtime.*
11 import androidx.compose.ui.Alignment
12 import androidx.compose.ui.Modifier
13 import androidx.compose.ui.graphics.Color
14 import androidx.compose.ui.res.painterResource
15 import androidx.compose.ui.unit.dp
16 import androidx.compose.ui.unit.sp
17
18 class MainActivityCompose : ComponentActivity() {
19     override fun onCreate(savedInstanceState:
20 Bundle?) {
21         super.onCreate(savedInstanceState)
22         setContent {
23             DiceApp()
24         }
25     }
26
27     @Composable
28     fun DiceApp() {
29         var dice1 by remember { mutableStateOf(1) }
30         var dice2 by remember { mutableStateOf(1) }
31         var resultText by remember { mutableStateOf("Klik
32 Roll") }
33
34         Column(
35             modifier = Modifier
36                 .fillMaxSize()
37                 .background(Color(0xFF0D0B14)),
38             horizontalAlignment =
39 Alignment.CenterHorizontally,
40             verticalArrangement = Arrangement.Center
41         ) {
42             // DADU
43             Row(
44                 horizontalArrangement =
45 Arrangement.spacedBy(16.dp)
46             ) {
47                 Image(
48                     painter = painterResource(id =
49 getDiceImage(dice1)),
50                     contentDescription = "Dice 1",
51                     modifier = Modifier.size(190.dp)
52                 )
53             }
54         }
55     }
56 }

```

```

49         )
50
51         Image(
52             painter = painterResource(id =
53             getDiceImage(dice2)),
54             contentDescription = "Dice 2",
55             modifier = Modifier.size(190.dp)
56         )
57     }
58     Spacer(modifier = Modifier.height(20.dp))
59
60     // BUTTON
61     Button(
62         onClick = {
63             dice1 = (1..6).random()
64             dice2 = (1..6).random()
65
66             resultText = if (dice1 == dice2) {
67                 "Selamat, anda dapat dadu
double!"
68             } else {
69                 "Anda belum beruntung!"
70             }
71         },
72         colors = ButtonDefaults.buttonColors(
73             containerColor = Color(0xFF824CC9)
74         )
75     ) {
76         Text(
77             text = "Roll",
78             color = Color.Black,
79             fontSize = 18.sp
80         )
81     }
82
83     Spacer(modifier = Modifier.height(8.dp))
84
85     // HASIL
86     Text(
87         text = resultText,
88         color = Color.White,
89         fontSize = 18.sp
90     )
91 }

```

```

92     }
93
94     // FUNCTION GAMBAR DADU
95     fun getDiceImage(value: Int): Int {
96         return when (value) {
97             1 -> R.drawable.dice_1
98             2 -> R.drawable.dice_2
99             3 -> R.drawable.dice_3
100            4 -> R.drawable.dice_4
101            5 -> R.drawable.dice_5
102            else -> R.drawable.dice_6
103        }
104    }

```

### Activity\_Main.xml

*Tabel 3. Source Code Activity\_Main.XML Soal 1*

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3
4      xmlns:android="http://schemas.android.com/apk/res/and
roid"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      android:orientation="vertical"
8      android:background="#0D0B14"
9      android:padding="20dp"
10     android:gravity="center_horizontal">
11
12     <!-- SPACER ATAS -->
13     <View
14         android:layout_width="match_parent"
15         android:layout_height="0dp"
16         android:layout_weight="1"/>
17
18     <!-- DADU -->
19     <LinearLayout
20         android:layout_width="wrap_content"
21         android:layout_height="wrap_content"
22         android:orientation="horizontal"
23         android:layout_marginBottom="20dp"
24         android:gravity="center">

```

|    |                                            |
|----|--------------------------------------------|
| 25 | <ImageView                                 |
| 26 | android:id="@+id/dice1"                    |
| 27 | android:layout_width="220dp"               |
| 28 | android:layout_height="220dp"              |
| 29 | android:contentDescription="@string/dice1" |
| 30 | android:src="@drawable/dice_1" />          |
| 31 |                                            |
| 32 | <ImageView                                 |
| 33 | android:id="@+id/dice2"                    |
| 34 | android:layout_width="220dp"               |
| 35 | android:layout_height="220dp"              |
| 36 | android:contentDescription="@string/dice2" |
| 37 | android:src="@drawable/dice_1" />          |
| 38 | </LinearLayout>                            |
| 39 |                                            |
| 40 | <!-- BUTTON -->                            |
| 41 | <Button                                    |
| 42 | android:id="@+id/btnRoll"                  |
| 43 | android:layout_width="wrap_content"        |
| 44 | android:layout_height="wrap_content"       |
| 45 | android:text="@string/roll"                |
| 46 | android:textColor="#000000"                |
| 47 | android:textSize="18sp"/>                  |
| 48 |                                            |
| 49 | <!-- SPACER BAWAH -->                      |
| 50 | <View                                      |
| 51 | android:layout_width="match_parent"        |
| 52 | android:layout_height="0dp"                |
| 53 | android:layout_weight="1"/>                |
| 54 |                                            |
| 55 | <!-- HASIL -->                             |
| 56 | <TextView                                  |
| 57 | android:id="@+id/tvResult"                 |
| 58 | android:layout_width="wrap_content"        |
| 59 | android:layout_height="wrap_content"       |
| 60 | android:background="#FFFFFF"               |
| 61 | android:padding="8dp"                      |
| 62 | android:text="@string/klik_roll"           |
| 63 | android:textColor="#000000"                |
| 64 | android:textSize="20sp"/>                  |
| 65 |                                            |
| 66 | </LinearLayout>                            |

## B. Output Program



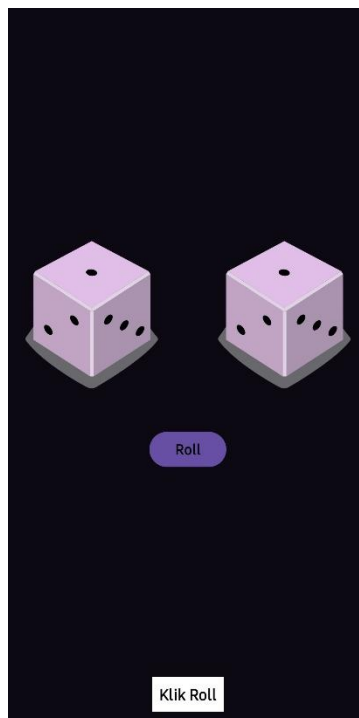
*Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose*



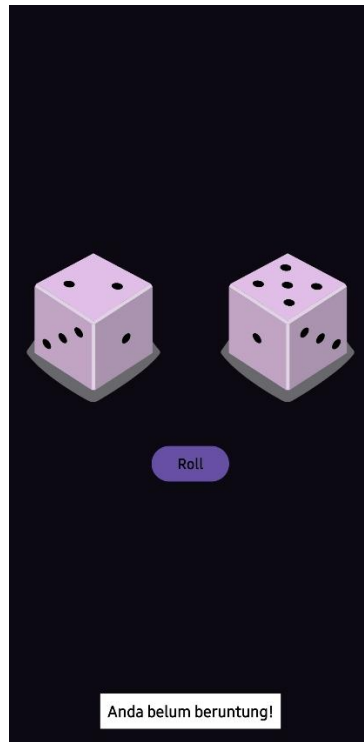
*Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose*



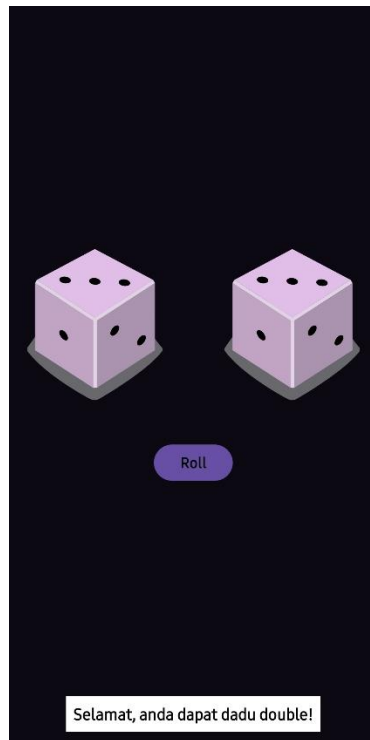
*Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose*



*Gambar 7. Screenshot Hasil Jawaban Soal 1 XML*



Gambar 8. Screenshot Hasil Jawaban Soal 1 XML



Gambar 9. Screenshot Hasil Jawaban Soal 1 XML

## C. Pembahasan

### MainActivity.kt XML

Kode ini untuk aplikasi sederhana lempar dadu menggunakan Kotlin dengan tampilan berbasis XML. Pada MainActivity, terdapat dua ImageView untuk menampilkan hasil dadu, satu Button sebagai tombol untuk menjalankan lemparan, serta satu TextView untuk menampilkan hasil dari permainan.

Pada metode `onCreate()`, layout `activity_main.xml` dipanggil menggunakan `setContentview()`. Setelah itu, setiap komponen dihubungkan ke kode menggunakan `findViewById()` sesuai dengan ID yang sudah dibuat di XML. Tombol `btnRoll` kemudian diberi aksi `setOnClickListener`, sehingga saat tombol ditekan akan memanggil fungsi `rollDice()`.

Fungsi `rollDice()` digunakan untuk menghasilkan dua angka acak dari 1 sampai 6 dengan `(1..6).random()` yang masing-masing mewakili hasil dari dua dadu. Setelah nilai didapat, fungsi `updateDiceImage()` dipanggil untuk memperbarui tampilan gambar pada `dice1` dan `dice2` sesuai hasil acakan. Selanjutnya dilakukan pengecekan, jika kedua nilai sama maka akan ditampilkan teks “Selamat, anda dapat dadu double!”, sedangkan jika berbeda akan muncul pesan “Anda belum beruntung!” pada TextView.

Fungsi `updateDiceImage()` berfungsi untuk mengatur gambar dadu berdasarkan nilai yang diterima. Proses pemilihan gambar menggunakan `when` untuk mencocokkan nilai 1 sampai 6 dengan resource gambar `dice_1` hingga `dice_6` yang sudah disimpan di folder `drawable`. Gambar yang sesuai kemudian ditampilkan menggunakan `setImageResource()`.

### MainActivity.kt Compose

Kode ini untuk aplikasi lempar dadu menggunakan Jetpack Compose pada Android. Pada bagian `MainActivityCompose`, class ini mewarisi `ComponentActivity` dan pada metode `onCreate()` dipanggil `setContent { DiceApp() }` untuk menampilkan tampilan utama aplikasi berbasis Compose.

Fungsi `DiceApp()` digunakan sebagai tampilan utama yang berisi state untuk menyimpan nilai dadu dan teks hasil. Terdapat tiga state yaitu `dice1` dan `dice2` untuk menyimpan angka dadu, serta `resultText` untuk menampilkan hasil permainan. Semua state ini menggunakan `remember` agar nilainya dapat berubah saat tombol ditekan.

Tampilan utama menggunakan `Column` dengan pengaturan full screen dan background warna gelap `0xFF0D0B14`. Di dalamnya terdapat `Row` yang menampilkan dua gambar dadu menggunakan `Image`, masing-masing mengambil resource dari fungsi `getDiceImage()` sesuai nilai dadu.

Di bawah gambar dadu terdapat `Button` dengan teks “Roll”. Saat tombol ditekan, sistem akan menghasilkan dua angka acak dari 1 sampai 6 menggunakan `(1..6).random()`, kemudian memperbarui nilai `dice1` dan `dice2`. Setelah itu dilakukan pengecekan, jika kedua nilai sama maka akan muncul teks “Selamat, anda dapat dadu double!”, jika tidak maka muncul “Anda belum beruntung!”.

Hasil ditampilkan melalui `Text` yang mengambil nilai dari `resultText`, sehingga setiap perubahan state langsung memperbarui tampilan secara otomatis. Fungsi `getDiceImage()` digunakan untuk menentukan gambar dadu berdasarkan nilai 1 sampai 6 menggunakan struktur `when`. Setiap nilai dipetakan ke resource drawable `dice_1` sampai `dice_6`, lalu dikembalikan sebagai referensi gambar yang ditampilkan pada `Image`.

### **Activity\_main.xml**

Kode XML ini adalah layout untuk aplikasi lempar dadu pada Android dengan menggunakan `LinearLayout` sebagai container utama. Layout dibuat secara vertikal dengan pengaturan background warna gelap `#0D0B14`, padding `20dp`, dan posisi konten berada di tengah secara horizontal.

Di bagian atas dan bawah terdapat `View` kosong dengan `layout_weight=1` yang berfungsi sebagai spacer agar posisi elemen utama berada di tengah layar. Bagian utama aplikasi berada di tengah berupa dua `ImageView` yang disusun secara horizontal di dalam `LinearLayout`. Kedua `ImageView` ini digunakan untuk menampilkan gambar dadu `dice1` dan `dice2` dengan ukuran masing-masing `220dp`, serta memiliki `contentDescription` dan default gambar `@drawable/dice_1`.

Di bawah tampilan dadu terdapat sebuah `Button` dengan id `btnRoll` yang digunakan untuk menjalankan proses lempar dadu. Tombol ini menampilkan teks dari string resource `@string/roll` dengan ukuran teks `18sp` dan warna hitam.

Pada bagian bawah terdapat `TextView` dengan id `tvResult` yang digunakan untuk menampilkan hasil dari lemparan dadu. `TextView` ini memiliki background putih, padding `8dp`, ukuran teks `20sp`, serta default teks `@string/klik_roll`.

## SOAL 2

Riset dan *cite* **artikel/jurnal/buku** mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatlah opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. \*\*

### A. Pembahasan

Dalam imperative programming, alur program dijelaskan lewat langkah-langkah yang jelas dan berurutan, termasuk bagaimana setiap proses dijalankan oleh programmer. Sedangkan dalam declarative programming, programmer hanya fokus pada hasil yang ingin dicapai tanpa harus menuliskan langkah-langkah detail, karena prosesnya akan diatur oleh sistem berdasarkan aturan yang diberikan (Fahland et al., 2009).

Perbedaan keduanya juga terlihat pada pengelolaan UI. Pada pendekatan imperative, perubahan komponen UI diatur secara langsung dan satu per satu oleh programmer, sedangkan pada declarative cukup menjelaskan tampilan akhirnya saja. Dalam pengembangan Android, imperative biasanya menggunakan Kotlin atau Java dengan XML, sementara declarative menggunakan Kotlin dengan Jetpack Compose yang lebih ringkas dan berbasis komponen (Nurhadi & Muslim, 2024).

Berdasarkan riset tersebut, saya lebih memilih imperative programming karena alurnya lebih mudah diikuti. Setiap proses ditulis secara jelas sehingga lebih gampang dipahami, terutama saat masih belajar dasar pemrograman. Cara ini juga membantu memahami bagaimana sistem bekerja dari awal sebelum masuk ke pendekatan yang lebih sederhana seperti declarative.

### SOAL 3

Apa yang membedakan OOP pada kotlin dengan OOP pada java? \*

#### A. Pembahasan

Kotlin dan Java sama-sama menerapkan paradigma pemrograman berorientasi objek (OOP), namun Kotlin memiliki pendekatan yang lebih modern karena juga mendukung pemrograman fungsional. Dibandingkan Java, Kotlin memiliki sintaks yang lebih ringkas sehingga penulisan kode menjadi lebih sederhana dan mudah dibaca. Selain itu, Kotlin menyediakan fitur null safety yang dapat mengurangi risiko terjadinya NullPointerException, serta fitur seperti extension function dan smart casting yang tidak tersedia pada Java. Java sendiri masih menggunakan pendekatan OOP yang lebih tradisional sehingga struktur kodenya cenderung lebih panjang dan eksplisit. Sehingga, Kotlin dianggap lebih efisien dan produktif dalam pengembangan aplikasi modern dibandingkan Java (Gotseva, Tomov, & Danov, 2019).

## SOAL 4

Sebutkan, dan jelaskan mengenai SOLID Principles \*

### A. Pembahasan

SOLID Principles merupakan kumpulan prinsip desain dalam pemrograman berorientasi objek yang terdiri dari Single Responsibility Principle, Open Closed Principle, Liskov Substitution Principle, Interface Segregation Principle, dan Dependency Inversion Principle. Prinsip SOLID ini digunakan untuk membantu pengembang menghasilkan desain perangkat lunak yang lebih baik, terutama agar kode tidak saling bergantung secara berlebihan dan lebih mudah dikembangkan di masa depan (Thennakoon & Hettige, 2021).

Single Responsibility Principle menjelaskan bahwa setiap kelas sebaiknya hanya memiliki satu tanggung jawab utama agar perubahan pada satu bagian tidak mempengaruhi bagian lain secara berlebihan. Open Closed Principle menekankan bahwa kode sebaiknya terbuka untuk pengembangan fitur baru, tetapi tidak perlu mengubah kode yang sudah ada agar sistem tetap stabil. Liskov Substitution Principle menjelaskan bahwa objek dari subclass harus bisa menggantikan superclass tanpa mengubah perilaku program. Interface Segregation Principle menekankan bahwa sebuah interface sebaiknya tidak terlalu besar, sehingga kelas tidak dipaksa menggunakan method yang tidak diperlukan. Terakhir, Dependency Inversion Principle menjelaskan bahwa modul tingkat tinggi sebaiknya bergantung pada abstraksi, bukan pada implementasi langsung, sehingga sistem lebih fleksibel dan mudah diubah (Thennakoon & Hettige, 2021).

## SOAL 5

Apa perbedaan antara Class dan Data Class pada Kotlin ?

### A. Pembahasan

Dalam Kotlin, class biasa dan data class sama-sama digunakan untuk membuat objek, tetapi kegunaannya berbeda. Class biasa biasanya dipakai untuk objek yang tidak hanya menyimpan data, tetapi juga memiliki proses atau logika tertentu. Pada class biasa, programmer perlu menuliskan sendiri fungsi-fungsi tambahan jika dibutuhkan, seperti `toString()`, `equals()`, atau `hashCode()`, sehingga lebih fleksibel tetapi juga lebih panjang dalam penulisannya. Sedangkan data class digunakan khusus untuk menyimpan data saja. Kotlin sudah menyediakan secara otomatis fungsi-fungsi penting seperti `toString()`, `equals()`, `hashCode()`, dan `copy()` tanpa perlu ditulis manual. Karena itu, data class lebih ringkas dan mudah digunakan ketika hanya fokus pada penyimpanan data, sedangkan class biasa lebih cocok untuk objek yang memiliki banyak aturan atau logika di dalamnya.

## DAFTAR PUSTAKA

- Fahland, D., Lübke, D., Mendling, J., Reijers, H., Weber, B., Weidlich, M., & Zugal, S. (2009, June). Declarative versus imperative process modeling languages: The issue of understandability. In *International Workshop on Business Process Modeling, Development and Support* (pp. 353-366). Berlin, Heidelberg: Springer Berlin Heidelberg.
- Nurhadi, N., & Muslim, A. (2025). Comparative Analysis of the Impact of Declarative and Imperative Programming Approaches on Android Application Performance Through Benchmarking Method. *Journal Research of Social Science, Economics & Management*, 5(5).
- Gotseva, D., Tomov, Y., & Danov, P. (2019, October). Comparative study java vs kotlin. In *2019 27th National Conference with International Participation (TELECOM)* (pp. 86-89). IEEE.
- Thennakoon, R. D. A. V., & Hettige, B. (2022). A study on object-oriented design principles and patterns.

## **TAUTAN GIT**

<https://github.com/nazlasalsabilaa/PemrogramanMobile.git>